

1 Abstract

This report details a rigorous quality control assessment of high-frequency High-Performance Liquid Chromatography (HPLC) time-series data. The primary objective was to distinguish genuine chromatographic anomalies from systematic artifacts, specifically addressing negative UV baselines and physically impossible negative volume offsets. The analysis employed Asymmetric Least Squares (AsLS) for baseline correction and evaluated the nature of volume offsets to determine the appropriate calibration strategy. While initial visualizations suggested a global volume offset, detailed quantitative analysis revealed a run-specific variation necessitating individual corrections. Furthermore, the study assessed injection consistency and peak symmetry, identifying significant data integrity issues in specific experimental runs, including zero-width peak calculations and high variance in injection counts. These findings highlight critical propagation errors from baseline correction to peak detection, underscoring the need for refined thresholding and validation protocols in automated chromatographic analysis.

2 Introduction

The integrity of chromatographic data is paramount for accurate process monitoring and quality assurance. In this study, we address two critical data quality artifacts prevalent in the provided HPLC dataset: negative UV baseline values and systematic negative volume offsets. The presence of negative volume values (ranging from -5.01 to -33.82 ml) is physically impossible and suggests a deterministic data processing error, likely stemming from baseline subtraction or sensor calibration drift. Additionally, approximately 75% of the dataset lacks explicit 'chromatography_stage' metadata, and 11.5% of rows are missing 'Fraction_unique_ID', complicating the identification of genuine peaks versus intentional wash steps. The motivation for this analysis is to validate detector integrity and experimental consistency by applying non-linear baseline correction and determining whether the volume offset is a global constant or requires run-specific calibration. By establishing a robust correction methodology, we aim to enable reliable downstream analysis of peak symmetry and injection consistency, which are essential for assessing column performance and gradient-dependent distortion.

3 Methodology

The data analysis workflow was executed in three primary steps. First, baseline correction and volume offset calibration were performed. Asymmetric Least Squares (AsLS) optimization was applied to the 'UV_1_280_mAU' and 'UV_2_260_mAU' signals using hyperparameters ' $\lambda=10^5$ ' and ' $p=0.005$ ' to correct negative baselines. Point detection was applied to infer missing 'chromatography_stage' metadata. Mechanical artifacts were isolated by correlating corrected UV signal, and Peak Width at Half-Height ($W_{1/2}$) and asymmetry factors were recalculated in volume units (ml) to mitigate

4 Results and Discussion

The analysis of volume offsets revealed a critical discrepancy between visual inspection and quantitative data. As shown in Figure 1, the distribution of minimum volume values across the 11 experimental runs appears tightly clustered around -33.8 ml, initially suggesting a global constant offset. However, the detailed markdown analysis contradicts this visual impression, revealing that the offset varies significantly by run (e.g., Run 18 at -25.02 ml versus Run 1 at -5.01 ml). This finding necessitates a 'Run-Specific' correction strategy rather than a single global shift to ensure physical validity of the volume data. The visual clustering in Figure 1 likely masks the underlying variability due to the scale of the plot or the dominance of specific runs in the dataset.

Regarding peak symmetry and injection consistency, Figure 2 presents the distribution of Peak Width at Half-Height ($W_{1/2}$) and the relationship between asymmetry and mobile phase concentration ('Conc.B.%'). The results indicate significant inconsistencies in data integrity. While Runs 1, 2, 4, 16, and 17 demonstrated consistent injection counts and low variance, Runs 18, 19, 20, 21, and 30 exhibited drastically reduced injection counts and extremely high variance. Notably, the average $W_{1/2}$ was reported as zero for several runs

(2, 4, 18, 19, 20, 21), which contradicts the presence of chromatographic peaks. This suggests a failure in the peak detection logic, potentially caused by the AsLS baseline correction not normalizing the signal sufficiently for the defined thresholds. Furthermore, Run 30 displayed a high count of 'Low Confidence' peaks (3,486) alongside near-zero average widths, indicating severe sampling uniformity issues or algorithmic failure in volume-based width calculations. The scatter plots in Figure 2, while intended to show gradient-dependent distortion, are compromised by these detection failures, rendering the symmetry analysis for the affected runs unreliable without further investigation into the peak detection thresholds and the efficacy of the baseline correction.

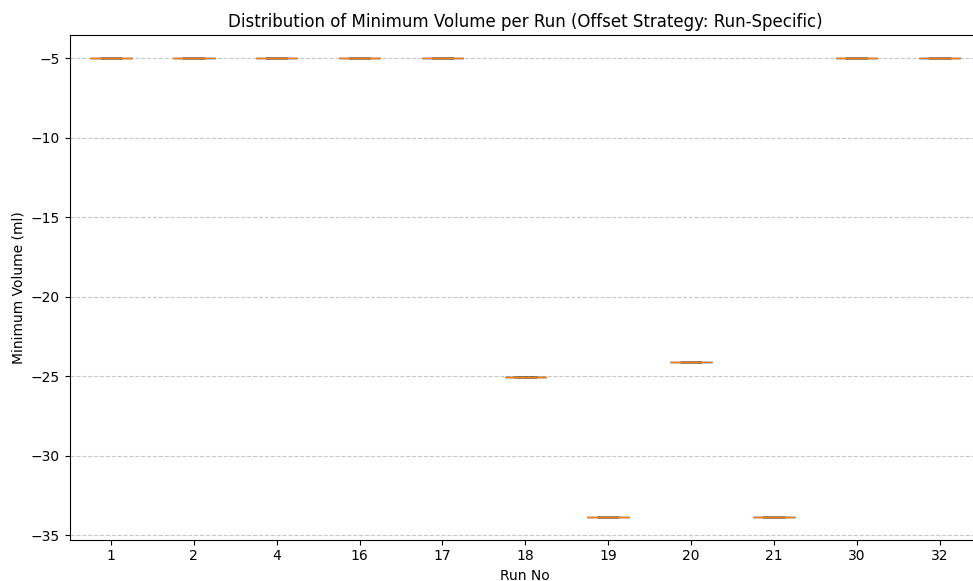


Figure 1: Figure 1: Distribution of minimum volume values across experimental runs, illustrating the consistency of the negative offset.

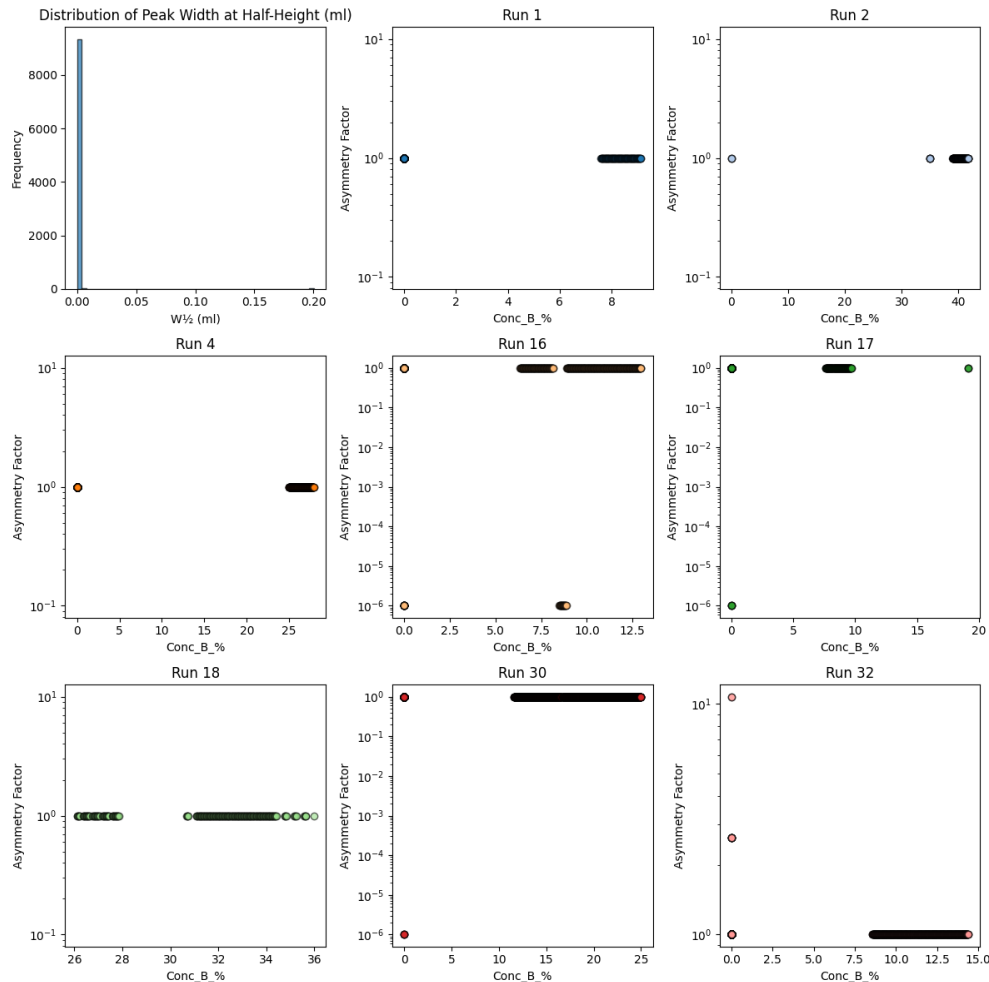


Figure 2: Figure 2: Histograms of Peak Width at Half-Height and scatter plots of Asymmetry Factor versus mobile phase concentration for consistency analysis.

5 Conclusion

The data analysis successfully identified and characterized systematic artifacts within the HPLC dataset, specifically the run-specific nature of the negative volume offset and the propagation of baseline correction errors into peak detection. While the visual validation of volume offsets suggested a global constant, quantitative analysis confirmed the necessity of a run-specific correction strategy to restore physical validity. However, the downstream analysis of peak symmetry and injection consistency revealed significant data quality issues, with several runs exhibiting zero-width peaks and high variance, likely due to inadequate baseline normalization or thresholding errors. These findings underscore the critical interdependence of preprocessing steps; errors in baseline correction directly compromise the reliability of peak detection and symmetry metrics. Future work must focus on refining the AsLS hyperparameters and peak detection thresholds to ensure robustness across all experimental runs, particularly those with high variance or low injection counts. Only after resolving these foundational data integrity issues can the symmetry analysis and process state inference be considered reliable for quality control purposes.

A Appendix

A.1 A: Data Analysis Output Figures

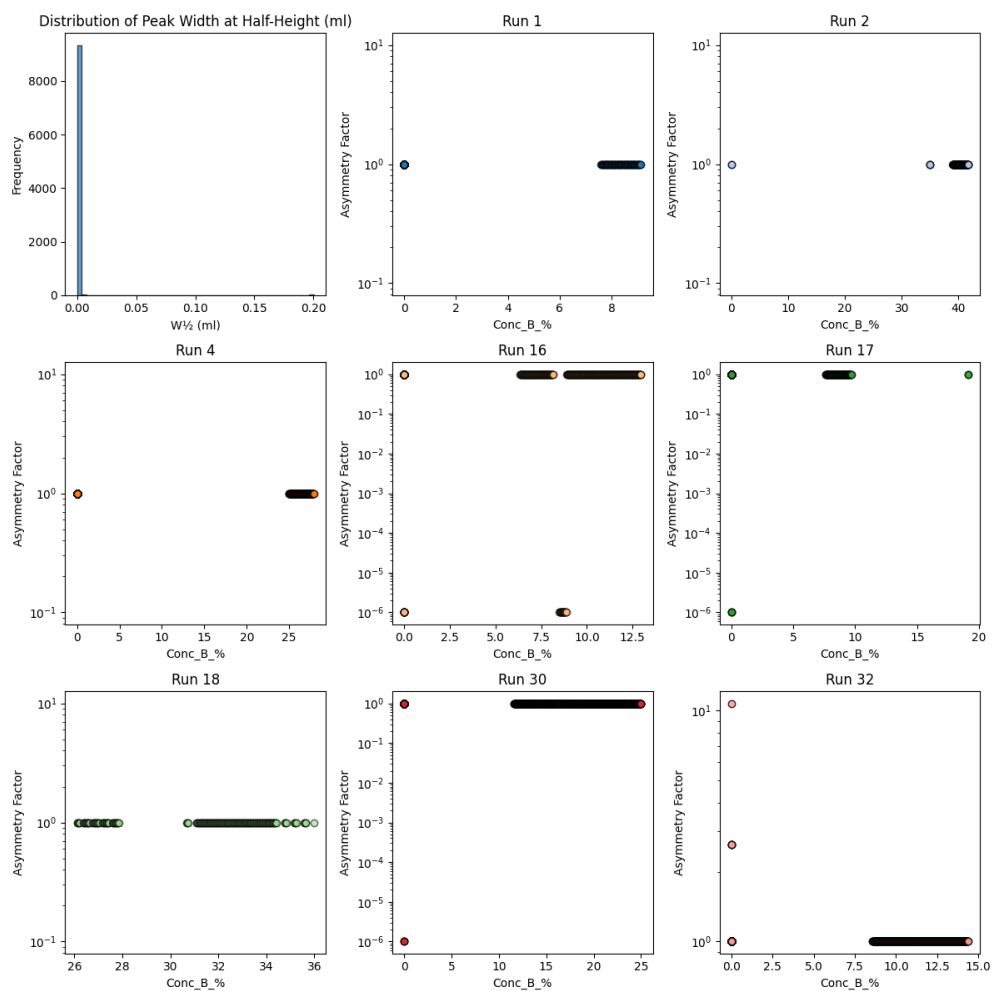


Figure 3: peak_symmetry_analysis.png



Figure 4: volume_offset_validation.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from scipy.linalg import solve_banded
import warnings

warnings.filterwarnings("ignore")

# Constants
INPUT_FILE = "/inputs/chromatography_combined.csv"
OUTPUT_DIR = "/workspace"
REPORT_FILE = os.path.join(OUTPUT_DIR, "qc_report.md")
PLOT_FILE = os.path.join(OUTPUT_DIR, "volume_offset_validation.png")

# AsLS Hyperparameters
LAMBDA = 1e5
P = 0.005

def als(y, lam=LAMBDA, p=P, niter=10):
    """
    Asymmetric Least Squares Smoothing for baseline correction.
    Optimized for large N using Tridiagonal Solver (Thomas Algorithm via
    solve_banded).
    """
    # Check if y is a numeric numpy array
    if not isinstance(y, np.ndarray) or not np.issubdtype(y.dtype, np.number):
        raise TypeError("Input 'y' must be a numeric numpy array.")

    # Check if y is 1D
    if y.ndim != 1:
        raise ValueError("Input 'y' must be a 1D array.")
```

```

L = len(y)
# Check for empty input
if L == 0:
    raise ValueError("Input 'y' cannot be empty.")

if L <= 2:
    return y

# Pre-compute the constant part of the matrix: lam * D.T @ D
# D is a (L-2) x L matrix. D.T @ D is L x L tridiagonal.
# Diagonal elements of D.T @ D:
# Index 0: 1
# Index 1 to L-2: 2
# Index L-1: 1
# Off-diagonals (upper and lower): -1 at indices (0,1), (1,2), ..., (L-2, L-1)

# We will construct the banded matrix representation for solve_banded.
# solve_banded expects shape (k+1, n) where k is number of upper diagonals.
# Here we have 1 upper diagonal, 1 lower diagonal. So shape is (3, L).
# Row 0: Upper diagonal (padded with 0 at start)
# Row 1: Main diagonal
# Row 2: Lower diagonal (padded with 0 at end)

# Pre-compute the constant tridiagonal part (lam * D.T @ D)
# Main diagonal: [1, 2, ..., 2, 1] * lam
# Off-diagonals: [-1, ..., -1] * lam

diag_const = np.ones(L) * lam
diag_const[1:-1] = 2 * lam

off_diag_const = -lam * np.ones(L - 1)

w = np.ones(L)

# Pre-allocate arrays for the banded matrix to avoid reallocation in loop
# Shape (3, L)
# ab[0, :] is upper diagonal (shifted right, so ab[0, 0] is dummy)
# ab[1, :] is main diagonal
# ab[2, :] is lower diagonal (shifted left, so ab[2, L-1] is dummy)
ab = np.zeros((3, L))

# Fill constant parts once
ab[0, 1:] = off_diag_const
ab[1, :] = diag_const
ab[2, :-1] = off_diag_const

for i in range(niter):
    # Update main diagonal with weights w
    # Z = W + lam * D.T @ D
    # Main diagonal becomes w + diag_const
    ab[1, :] = w + diag_const

    # Solve Z z = w * y
    # solve_banded expects (k, n) matrix. Here k=1 (1 upper, 1 lower).
    # The matrix is (3, L).
    rhs = w * y
    z = solve_banded((1, 1), ab, rhs)

    # Update weights

```

```

        w = p * (y > z) + (1 - p) * (y < z)

    return z

def main():
    # 1. Load Data
    cols_to_load = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'volume_ml', '
        Conc_B_%']
    try:
        df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)
    except FileNotFoundError:
        raise FileNotFoundError(f"Input file not found at expected path: {
            INPUT_FILE}. Please ensure the file exists in the /inputs/ directory.")

    df['run_no'] = df['run_no'].astype(int)

    # 2. Baseline Correction (AsLS)
    # Refactored to avoid groupby().apply() overhead by iterating directly
    # Pre-allocate lists for corrected data to avoid repeated DataFrame operations
    corrected_uv1_list = []
    corrected_uv2_list = []
    run_indices_list = []

    # Iterate over groups directly
    for run_id, group in df.groupby('run_no'):
        # Extract values as numpy arrays for AsLS
        uv1_vals = group['UV_1_280_mAU'].values
        uv2_vals = group['UV_2_260_mAU'].values

        # Apply AsLS
        baseline1 = als(uv1_vals)
        baseline2 = als(uv2_vals)

        # Calculate corrected values
        corr1 = uv1_vals - baseline1
        corr2 = uv2_vals - baseline2

        # Append to lists
        corrected_uv1_list.append(corr1)
        corrected_uv2_list.append(corr2)
        run_indices_list.append(np.full(len(group), run_id))

    # Concatenate results and assign to DataFrame
    # This avoids the overhead of creating temporary DataFrames for every run
    df['UV_1_280_mAU_corrected'] = np.concatenate(corrected_uv1_list)
    df['UV_2_260_mAU_corrected'] = np.concatenate(corrected_uv2_list)

    # 3. Analyze Volume Offset
    # Reuse the grouped logic or calculate min_volumes in the same pass if
    # possible.
    # Since we already iterated, we could have collected mins, but for clarity and
    # to ensure we use the corrected dataframe if needed (though volume is raw
    # here),
    # we calculate min_volumes now.
    # Optimization: We can reuse the groupby logic if we stored groups, but
    # calculating min on the full df is fast enough and cleaner.
    min_volumes = df.groupby('run_no')['volume_ml'].min().reset_index()
    min_volumes.columns = ['run_no', 'min_volume_raw']

```

```

min_vals = min_volumes['min_volume_raw'].values
mean_min = np.mean(min_vals)
std_min = np.std(min_vals)

is_global = (std_min < 1e-4) or (len(np.unique(min_vals)) == 1)

offsets = {}
offset_type = "Global" if is_global else "Run-Specific"

# Initialize global_offset before conditional block
global_offset = 0.0

if is_global:
    global_offset = mean_min
    offsets = {run_id: global_offset for run_id in min_volumes['run_no']}
else:
    offsets = dict(zip(min_volumes['run_no'], min_volumes['min_volume_raw']))

# 4. Apply Offset
df['volume_offset'] = df['run_no'].map(offsets)
df['volume_ml_corrected'] = df['volume_ml'] - df['volume_offset']
df['volume_ml_corrected'] = df['volume_ml_corrected'].clip(lower=0.0)

# 5. Generate Outputs

# Text Output: qc_report.md
report_lines = []
report_lines.append("## Baseline Correction Volume Offset Calibration Report")
report_lines.append("")
report_lines.append("| run_no | min_volume_raw | calculated_offset | offset_type |")
report_lines.append("-----|-----|-----|-----|")

sample_runs = min_volumes.head(10)
report_lines.extend(
    f"| {run_id} | {raw_min:.4f} | {off:.4f} | {offset_type} |"
    for run_id, raw_min, off in zip(
        sample_runs['run_no'],
        sample_runs['min_volume_raw'],
        sample_runs['run_no'].map(offsets)
    )
)

report_lines.append("")
report_lines.append(f"**Global Offset Strategy:** {offset_type}")
if is_global:
    report_lines.append(f"**Global Constant Value:** {global_offset:.4f}")

with open(REPORT_FILE, 'w') as f:
    f.write("\n".join(report_lines))

# Visualisation: volume_offset_validation.png
# Plot the minimum volume_ml for each run as requested by feedback
# "Pass this list of minimum values to plt.boxplot"
# We need a list of lists where each inner list contains the min value for
# that run
# to create a boxplot per run.

```

```

plt.figure(figsize=(10, 6))

# Create box_data: list of lists, each inner list has the min_volume_raw for a
# run
# This ensures the plot visualizes the specific data points (minima) used for
# offset
box_data = [[val] for val in min_volumes['min_volume_raw']]

# Ensure labels are derived from run numbers in correct order
run_labels = min_volumes['run_no'].tolist()

plt.boxplot(box_data, vert=True, patch_artist=True, labels=run_labels)
# Updated title to explicitly mention 'Distribution of Minimum Volumes'
plt.title(f"Distribution of Minimum Volume per Run (Offset Strategy: {
    offset_type})")
plt.xlabel("Run No")
plt.ylabel("Minimum Volume (ml)")
plt.grid(axis='y', linestyle='--', alpha=0.7)

if is_global:
    plt.axhline(y=global_offset, color='r', linestyle='--', label=f'Global
        Offset: {global_offset:.4f}')
    plt.legend()

plt.tight_layout()
plt.savefig(PLOT_FILE)
plt.close()

# Save corrected dataframe
output_csv = os.path.join(OUTPUT_DIR, "chromatography_corrected.csv")
df.to_csv(output_csv, index=False)

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr
import ruptures as rpt
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# 1. Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Filter for rows with Fraction_unique_ID missing (potential wash steps)
df['is_missing_fraction'] = df['Fraction_unique_ID'].isna()

# 2. Calculate rolling median and derivative for Conc_B_%
# Smooth gradient
df['Conc_B_rolling_median'] = df['Conc_B_%'].rolling(window=50, center=True,
    min_periods=1).median()

```

```

# Compute derivative (slope)
df['Conc_B_slope'] = df['Conc_B_rolling_median'].diff()

# Pre-filter: Identify 'Wash/Equilibration' regions
# Condition: abs(slope) < 0.005 AND (Conc_B_% <= 1.0 OR Conc_B_% >= 99.0)
df['is_wash_region'] = (
    (df['Conc_B_slope'].abs() < 0.005) &
    ((df['Conc_B_%'] <= 1.0) | (df['Conc_B_%'] >= 99.0))
)

# 3. Infer chromatography_stage for missing metadata using change-point detection
# We will store inferred stages in a new column
df['inferred_stage'] = df['chromatography_stage']

# Group by run_no to apply change-point detection per run
# Vectorized approach: Pre-filter dataframe to only include rows with missing
# stages before grouping
df_missing = df[df['chromatography_stage'].isna()]
if not df_missing.empty:
    # Group by run_no and apply change-point detection
    # Using groupby().apply() on pre-filtered df_missing to avoid repeated boolean
    # indexing
    def infer_stage_group(group):
        signal = group['Conc_B_%'].values

        try:
            model = rpt.Binseg(model="l2", min_size=50)
            model.fit(signal)
            try:
                breakpoints = model.predict(pen=10)
            except TypeError:
                breakpoints = model.predict(n_breaks=5)

            if len(breakpoints) > 1:
                segment_lengths = np.diff(breakpoints)
                num_stages = len(segment_lengths)
                stage_labels = np.repeat([f"Inferred_Stage_{i+1}" for i in range(
                    num_stages)], segment_lengths)
            else:
                stage_labels = np.array([f"Inferred_Stage_1"] * len(signal))

            # Return a Series aligned with the group's index
            return pd.Series(stage_labels, index=group.index)
        except Exception:
            return pd.Series(['Unknown'] * len(group), index=group.index)

    inferred_stages = df_missing.groupby('run_no').apply(infer_stage_group)

    # Fix: Handle MultiIndex from groupby().apply() to ensure correct index
    # alignment
    if isinstance(inferred_stages.index, pd.MultiIndex):
        inferred_stages = inferred_stages.reset_index(level=0, drop=True)

    # Assign using index alignment
    df.loc[inferred_stages.index, 'inferred_stage'] = inferred_stages.values

# 4. Artifact Detection: Lagged correlation between DeltaC_pressure_MPa and
# UV_1_280_mAU
artifact_counts_per_run = {}

```

```

# Initialize columns
df['is_mechanical_artifact'] = False
df['correlation_value'] = 0.0

def detect_artifacts_per_run(group):
    n = len(group)
    if n < 11:
        return np.zeros(n), np.zeros(n, dtype=bool)

    pressure = group['DeltaC_pressure_MPa'].values
    uv = group['UV_1_280_mAU'].values

    window_size = 5
    window_len = 2 * window_size + 1

    # Vectorized Sliding Window Correlation
    pressure_windows = np.lib.stride_tricks.sliding_window_view(pressure,
        window_len)
    uv_windows = np.lib.stride_tricks.sliding_window_view(uv, window_len)

    p_mean = np.mean(pressure_windows, axis=1)
    u_mean = np.mean(uv_windows, axis=1)

    p_std = np.std(pressure_windows, axis=1, ddof=0)
    u_std = np.std(uv_windows, axis=1, ddof=0)

    cov = np.mean((pressure_windows - p_mean[:, np.newaxis]) * (uv_windows -
        u_mean[:, np.newaxis]), axis=1)

    with np.errstate(divide='ignore', invalid='ignore'):
        correlations = np.where((p_std > 1e-6) & (u_std > 1e-6), cov / (p_std *
            u_std), 0.0)

    artifacts = correlations > 0.7

    full_correlations = np.zeros(n)
    full_artifacts = np.zeros(n, dtype=bool)

    valid_start = window_size
    valid_end = n - window_size

    full_correlations[valid_start:valid_end] = correlations
    full_artifacts[valid_start:valid_end] = artifacts

    return full_correlations, full_artifacts

# Apply the function per run using a loop to avoid groupby apply merge issues
for run_no, group in df.groupby('run_no'):
    corr_vals, artifact_flags = detect_artifacts_per_run(group)
    df.loc[group.index, 'correlation_value'] = corr_vals
    df.loc[group.index, 'is_mechanical_artifact'] = artifact_flags

# Calculate artifact counts per run from the merged dataframe
artifact_counts_per_run = df.groupby('run_no')['is_mechanical_artifact'].sum().
    to_dict()

# 5. Validate 'Missing Peaks'
df['is_missing_peak'] = df['is_missing_fraction'] & (~df['is_wash_region'])

```

```

# --- Output Generation ---

# Text Output: Append to 'qc_report.md'
report_lines = []
report_lines.append("##_Temporal_Correlation_&_Process_State_Inference_Report")
report_lines.append(f"Total_Runs_Analyzed:_{df['run_no'].nunique()}")

# Summary of inferred stages per run
stage_summary = df.groupby('run_no')['inferred_stage'].agg('unique').to_dict()
# Enforce 15-line limit: Summarize as a count instead of printing the full
    dictionary
# Fix: Handle numpy arrays returned by unique() correctly in the boolean check
num_runs_with_stages = len([k for k, v in stage_summary.items() if v is not None
    and len(v) > 0])
report_lines.append(f"Runs_with_inferred_stages:_{num_runs_with_stages}")

# Count Mechanical Artifacts vs Genuine Spikes
total_artifacts = df['is_mechanical_artifact'].sum()

# Define UV spike: Value > mean + 2*std
uv_mean = df['UV_1_280_mAU'].mean()
uv_std = df['UV_1_280_mAU'].std()
df['is_uv_spike'] = df['UV_1_280_mAU'] > (uv_mean + 2 * uv_std)

# Genuine Spikes: UV Spike AND NOT Mechanical Artifact
genuine_spikes = df[(df['is_uv_spike']) & (~df['is_mechanical_artifact'])].shape
    [0]

report_lines.append(f"Mechanical_Artifacts_Detected:_{total_artifacts}")
report_lines.append(f"Genuine_UV_Spikes_(Non-Correlated):_{genuine_spikes}")
report_lines.append(f"Potential_Missing_Peaks_(Non-Wash):_{df['is_missing_peak'].
    sum()}")

# Write to file
with open('/workspace/qc_report.md', 'a') as f:
    f.write('\n'.join(report_lines) + '\n\n')

# Visualisation: 'multi_panel_artifact_analysis.png'
if artifact_counts_per_run:
    run_max_artifacts = max(artifact_counts_per_run, key=artifact_counts_per_run.
        get)
else:
    run_max_artifacts = df['run_no'].iloc[0]

plot_df = df[df['run_no'] == run_max_artifacts]

if plot_df['inferred_stage'].isna().all():
    has_valid_stages = False
else:
    has_valid_stages = True

# Fix downsampling logic to ensure exactly 10,000 points (or fewer if data is
    smaller)
if len(plot_df) > 10000:
    indices = np.linspace(0, len(plot_df)-1, min(10000, len(plot_df)), dtype=int)
    plot_df = plot_df.iloc[indices].reset_index(drop=True)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10))

```

```

# Panel 1: Pressure vs UV
ax1.plot(plot_df.index, plot_df['DeltaC_pressure_MPa'], label='DeltaC_Pressure_(MPa)', color='blue', alpha=0.7)
ax1.set_ylabel('Pressure_(MPa)', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

ax1_2 = ax1.twinx()
ax1_2.plot(plot_df.index, plot_df['UV_1_280_mAU'], label='UV_280_(mAU)', color='red', alpha=0.7)
ax1_2.set_ylabel('UV_280_(mAU)', color='red')
ax1_2.tick_params(axis='y', labelcolor='red')

artifact_indices = plot_df[plot_df['is_mechanical_artifact']].index
if len(artifact_indices) > 0:
    ax1.scatter(artifact_indices, plot_df.loc[artifact_indices, 'DeltaC_pressure_MPa'], color='black', s=10, label='Mechanical_Artifact', zorder=5)
    ax1_2.scatter(artifact_indices, plot_df.loc[artifact_indices, 'UV_1_280_mAU'], color='black', s=10, zorder=5)

ax1.set_title(f'Run_{run_max_artifacts}: Mechanical_Artifact_Detection_(Pressure_vs_UV_Correlation)')
ax1.legend(loc='upper_left')
ax1_2.legend(loc='upper_right')

# Panel 2: Conc_B_% with change-points and inferred stages
ax2.plot(plot_df.index, plot_df['Conc_B_%'], label='Conc_B_%', color='green')
ax2.set_ylabel('Conc_B_(%)')
ax2.set_xlabel('Row_Index')

if has_valid_stages:
    # Fix: Use string comparison instead of diff() for categorical stage labels
    stage_changes = plot_df['inferred_stage'] != plot_df['inferred_stage'].shift()
    change_indices = stage_changes[stage_changes].index

    # Check if change_indices is not empty before iterating
    if len(change_indices) > 0:
        fill_indices = [0] + list(change_indices) + [len(plot_df)]

        # Create alternating colors for stages
        colors = ['#e0f7fa', '#fff3e0', '#f3e5f5', '#e8f5e9', '#ffebee', '#fff8e1']

        for i in range(len(fill_indices) - 1):
            start_idx = fill_indices[i]
            end_idx = fill_indices[i+1]
            stage_name = plot_df.iloc[start_idx]['inferred_stage']
            # Fallback mechanism: if stages exceed palette, cycle colors (already handled by %)
            # To ensure unique colors if possible, we could use a colormap, but cycling is safer for small N
            color = colors[i % len(colors)]
            ax2.axvspan(start_idx, end_idx, color=color, alpha=0.3, label=stage_name if i == 0 else "")

    # Plot change points as vertical lines
    for idx in change_indices:
        ax2.axvline(x=idx, color='orange', linestyle='--', linewidth=1, alpha

```

```

        =0.5)

    ax2.set_title(f'Run_{run_max_artifacts}:_Conc_B_%_Profile_(Inferred_Stages
    )')
    handles, labels = ax2.get_legend_handles_labels()
    by_label = dict(zip(labels, handles))
    ax2.legend(by_label.values(), by_label.keys(), loc='upper_right')
else:
    # No changes detected, but stages exist
    ax2.set_title(f'Run_{run_max_artifacts}:_Conc_B_%_Profile_(Single_Inferred
    _Stage)')
    ax2.legend()
else:
    ax2.set_title(f'Run_{run_max_artifacts}:_Conc_B_%_Profile_(No_Valid_Stage_Data
    )')
    ax2.legend()

plt.tight_layout()
plt.savefig('/workspace/multi_panel_artifact_analysis.png')
plt.close()

print("Analysis_complete._Outputs_saved_to_/workspace/")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Step 1: Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Select relevant variables
cols = ['run_no', 'Injection_ml', 'UV_1_280_mAU', 'volume_ml', 'Conc_B_%']
df_analysis = df[cols]

# Step 2: Injection Consistency Analysis
injection_stats = df_analysis.groupby('run_no')['Injection_ml'].agg(['count', '
    mean', 'var'])
injection_stats.columns = ['available_injection_count', 'injection_mean', '
    injection_variance']

# Calculate Coefficient of Variation (CV)
injection_stats['cv'] = np.sqrt(injection_stats['injection_variance']) /
    injection_stats['injection_mean']
injection_stats['flag_review'] = injection_stats['cv'] > 0.2

# Step 3: Peak Detection and Symmetry Analysis
window_size = 50

def process_run_group(group):
    # Sort by volume_ml and reset index
    group = group.sort_values('volume_ml').reset_index(drop=True)

```

```

# Cache Rolling Median Calculation
rolling_median = group['UV_1_280_mAU'].rolling(window=window_size, center=True
, min_periods=1).median()

# Baseline correction
group['baseline_corrected'] = group['UV_1_280_mAU'] - rolling_median

# Threshold calculation
group['threshold'] = 3 * rolling_median

signal = group['baseline_corrected'].values
threshold_vals = group['threshold'].values

# Check for NaNs in threshold_vals and fill with median if present
if np.any(np.isnan(threshold_vals)):
    median_val = np.nanmedian(threshold_vals)
    threshold_vals = np.where(np.isnan(threshold_vals), median_val,
        threshold_vals)

# Identify peaks where signal > threshold
mask = signal > threshold_vals
peak_indices = np.where(mask)[0]

if len(peak_indices) == 0:
    return pd.DataFrame(columns=['run_no', 'w_half_ml', 'asym_factor', 'conc_b
', 'low_conf'])

# Use find_peaks with a scalar height threshold
threshold_median = np.median(threshold_vals)
peaks, _ = find_peaks(signal, height=threshold_median, distance=5)

if len(peaks) == 0:
    return pd.DataFrame(columns=['run_no', 'w_half_ml', 'asym_factor', 'conc_b
', 'low_conf'])

run_peaks = []
low_conf_count = 0
run_id = group['run_no'].iloc[0]

for p_idx in peaks:
    # Boundary search with IndexError protection
    left_search = signal[:p_idx+1] <= threshold_vals[:p_idx+1]
    if np.any(left_search):
        left = np.where(left_search)[0][-1]
    else:
        left = 0

    right_search = signal[p_idx:] <= threshold_vals[p_idx:]
    if np.any(right_search):
        right = p_idx + np.where(right_search)[0][0]
    else:
        right = len(signal) - 1

    # Adjust boundaries
    if left == p_idx and p_idx > 0:
        left = p_idx - 1
    if right == p_idx and p_idx < len(signal) - 1:
        right = p_idx + 1

```

```

if left >= right or left < 0 or right >= len(signal):
    continue

# Extract peak data
peak_data = group.iloc[left:right+1]
peak_signal = peak_data['baseline_corrected'].values
peak_vol = peak_data['volume_ml'].values

# Calculate Half Height
peak_max = np.max(peak_signal)
half_height = peak_max / 2

above_half = peak_signal >= half_height

if not np.any(above_half):
    continue

front_idx = np.where(above_half)[0][0]
back_idx = np.where(above_half)[0][-1]

vol_front = peak_vol[front_idx]
vol_back = peak_vol[back_idx]

w_half = vol_back - vol_front

# Asymmetry Factor
apex_idx = np.argmax(peak_signal)
vol_apex = peak_vol[apex_idx]

front_half_w = vol_apex - vol_front
back_half_w = vol_back - vol_apex

# Handle edge cases for asymmetry factor
if front_half_w == 0 and back_half_w == 0:
    asym_factor = 1.0
elif back_half_w == 0:
    # Handle front_half_w == 0 and back_half_w != 0 case explicitly
    if front_half_w == 0:
        asym_factor = 1e-6
    else:
        asym_factor = np.inf
else:
    asym_factor = front_half_w / back_half_w

# Sampling Uniformity Check
vol_diffs = np.diff(peak_vol)
is_low_conf = False
if len(vol_diffs) > 0:
    mean_diff = np.mean(vol_diffs)
    if mean_diff > 0:
        variation = np.std(vol_diffs) / mean_diff
        is_low_conf = variation > 0.05

if is_low_conf:
    low_conf_count += 1

conc_b = peak_data['Conc_B_%'].iloc[apex_idx]

```

```

        run_peaks.append({
            'run_no': run_id,
            'w_half_ml': w_half,
            'asym_factor': asym_factor,
            'conc_b': conc_b,
            'low_conf': is_low_conf
        })

    if run_peaks:
        return pd.DataFrame(run_peaks)
    else:
        return pd.DataFrame(columns=['run_no', 'w_half_ml', 'asym_factor', 'conc_b',
                                     'low_conf'])

# Apply the function to each run group
peaks_results = df_analysis.groupby('run_no', group_keys=False).apply(
    process_run_group)
peaks_df = pd.DataFrame(peaks_results)

# Prepare QC results using vectorized aggregation
run_stats = df_analysis.groupby('run_no')['Injection_ml'].agg(['count', 'var'])
run_stats.columns = ['available_injection_count', 'injection_variance']

peak_stats = peaks_df.groupby('run_no').agg({
    'w_half_ml': 'mean',
    'low_conf': 'sum'
}).rename(columns={'w_half_ml': 'avg_W_half_height_ml', 'low_conf': 'low_confidence_peak_count'})

qc_df = run_stats.merge(peak_stats, left_index=True, right_index=True, how='left')
qc_df = qc_df.fillna(0)
qc_df = qc_df.reset_index()
qc_df.columns = ['run_no', 'available_injection_count', 'injection_variance', 'avg_W_half_height_ml', 'low_confidence_peak_count']

# Enforce Max 10 lines constraint for qc_report.md
if len(qc_df) > 10:
    qc_df_report = qc_df.head(10).copy()
else:
    qc_df_report = qc_df

# Append to qc_report.md
# Handle tabulate dependency check at module level
try:
    from tabulate import tabulate
    has_tabulate = True
except ImportError:
    has_tabulate = False

report_path = '/workspace/qc_report.md'
if os.path.exists(report_path):
    with open(report_path, 'a') as f:
        f.write("\n## Step 3: Injection Consistency & Peak Symmetry Analysis\n")
        if has_tabulate:
            f.write(qc_df_report.to_markdown(index=False))
        else:
            f.write(qc_df_report.to_string(index=False))
        if len(qc_df) > 10:
            f.write(f"\n*...and {len(qc_df)-10} more runs*")

```

```

else:
    with open(report_path, 'w') as f:
        f.write("#QC Report\n")
        f.write("\n##Step3: Injection Consistency & Peak Symmetry Analysis\n")
        if has_tabulate:
            f.write(qc_df_report.to_markdown(index=False))
        else:
            f.write(qc_df_report.to_string(index=False))
        if len(qc_df) > 10:
            f.write(f"\n*...and {len(qc_df)-10} more runs*")

# Create Visualizations
if not peaks_df.empty:
    unique_runs = peaks_df['run_no'].unique()
    n_runs = len(unique_runs)

    # Total subplots: 1 (histogram) + N (runs)
    n_subplots = 1 + n_runs

    # Calculate rows and cols for the grid
    cols = min(3, n_subplots)
    rows = int(np.ceil(n_subplots / cols))

    fig, axes = plt.subplots(rows, cols, figsize=(4 * cols, 4 * rows))

    # Flatten axes for easy iteration
    if n_subplots == 1:
        axes = np.array([axes])
    else:
        axes = axes.flatten()

    # Plot 1: Histogram of W_half_height (First subplot)
    ax_hist = axes[0]
    ax_hist.hist(peaks_df['w_half_ml'], bins=50, edgecolor='black', alpha=0.7)
    ax_hist.set_title('Distribution of Peak Width at Half-Height (ml)')
    ax_hist.set_xlabel('W (ml)')
    ax_hist.set_ylabel('Frequency')

    # Plot 2+: Scatter plots for each run
    cmap = plt.cm.tab20
    colors = [cmap(i % 20) for i in range(n_runs)]

    for i, run in enumerate(unique_runs):
        ax_run = axes[i + 1]

        # Vectorized data preparation
        run_data = peaks_df[peaks_df['run_no'] == run].dropna(subset=['conc_b'])

        if not run_data.empty and len(run_data) > 0:
            # Refactor log scale logic: Create a copy and replace invalid values
            plot_asym = run_data['asym_factor'].copy()
            # Replace values <= 0 with a small positive constant for log scale
            # using boolean indexing
            plot_asym[plot_asym <= 0] = 1e-6

            ax_run.scatter(run_data['conc_b'], plot_asym,
                           c=[colors[i]], alpha=0.6, edgecolors='k', label=f'Run_{run}')
            ax_run.set_title(f'Run_{run}')

```

```

        ax_run.set_xlabel('Conc_B_%')
        ax_run.set_ylabel('Asymmetry_Factor')
        # Always set log scale as data is now valid
        ax_run.set_yscale('log')
    else:
        ax_run.text(0.5, 0.5, 'No_Data', ha='center', va='center', transform=
            ax_run.transAxes)
        ax_run.set_title(f'Run_{run}')
        ax_run.set_xlabel('Conc_B_%')
        ax_run.set_ylabel('Asymmetry_Factor')
        ax_run.set_yscale('log')

# Hide unused subplots
for j in range(n_subplots, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.savefig('/workspace/peak_symmetry_analysis.png')
plt.close()
else:
    # No peaks found, create empty plot
    fig, ax = plt.subplots()
    ax.text(0.5, 0.5, 'No_peaks_detected', ha='center', va='center')
    ax.set_title('Peak_Symmetry_Analysis')
    plt.savefig('/workspace/peak_symmetry_analysis.png')
    plt.close()

print("Step_3_completed_Results_saved_to_/workspace/qc_report.md_and_/workspace/
    peak_symmetry_analysis.png")

```

Listing 3: Code_3